

Musterlösung: Interfaces in Java

1. Interface Druckbar

```
package template.drucken;

public interface Druckbar
{
    String DINA4 = "297 mm x 210 mm";

    void drucken();

    static void testDruck()
    {
        System.out.println("Druckergeräusche... es  
wird ein Testblatt gedruckt.");
    }

    default boolean prüfeFormat(String format)
    {
        return format.equalsIgnoreCase(DINA4);
    }
}
```

Erläuterung

- Druckbar beschreibt die Fähigkeit, dass ein Objekt gedruckt werden kann.
- DINA4 ist eine Konstante.
- drucken() ist eine abstrakte Methode und muss von implementierenden Klassen umgesetzt werden.
- testDruck() ist eine statische Methode und wird direkt über das Interface aufgerufen.
- prüfeFormat(...) ist eine default-Methode und kann von

implementierenden Klassen direkt genutzt werden.

2. Klasse Dokument

```
package template.drucken;

public class Dokument implements Druckbar
{
    protected String text;

    public Dokument(String text)
    {
        this.text = text;
    }

    public String getText()
    {
        return text;
    }

    public void setText(String text)
    {
        this.text = text;
    }

    @Override
    public void drucken()
    {
        System.out.println("Druckergeräusche... Das
Dokument wird gedruckt: " + text);
    }
}
```

Erläuterung

- Dokument implementiert das Interface Druckbar.
- Deshalb muss die Methode drucken () konkret umgesetzt werden.

- Das Attribut text speichert den Inhalt des Dokuments.
-

3. Interface PDFDruckbar

```
package template.drucken;

public interface PDFDruckbar extends Druckbar
{
    void pdfDrucken();
}
```

Erläuterung

- PDFDruckbar erweitert Druckbar.
 - Damit übernimmt es auch die Methode drucken().
 - Zusätzlich fordert es die Methode pdfDrucken().
-

4. Interface Mailbar

```
package template.drucken;

public interface Mailbar
{
    void mailVersenden();
}
```

Erläuterung

- Mailbar beschreibt die Fähigkeit, ein Objekt per Mail zu versenden.
-

5. Klasse PDFDokument

```
package template.drucken;

public class PDFDokument extends Dokument implements
PDFDruckbar, Mailbar
{
    public PDFDokument(String text)
    {
        super(text);
    }

    @Override
    public void pdfDrucken()
    {
        System.out.println("Digitale
Druckergeräusche... Das Dokument wird als PDF
gedruckt: " + text);
    }

    @Override
    public void mailVersenden()
    {
        System.out.println("Digitale
Briefgeräusche... Das Dokument wird per Mail
versendet: " + text);
    }
}
```

Erläuterung

- PDFDokument erbt von Dokument.
- Dadurch übernimmt es bereits die Methode drucken().
- Zusätzlich implementiert es PDFDruckbar und Mailbar.
- Deshalb muss es pdfDrucken() und mailVersenden() umsetzen.

6. Interface HatFläche

```
package template.fläche;

public interface HatFläche
{
    double berechneFläche();
}
```

Erläuterung

- HatFläche beschreibt die Fähigkeit, eine Fläche zu berechnen.
-

7. Klasse Kreis

```
package template.fläche;

public class Kreis implements HatFläche
{
    private double radius;

    public Kreis(double radius)
    {
        this.radius = radius;
    }

    public double getRadius()
    {
        return radius;
    }

    public void setRadius(double radius)
    {
        this.radius = radius;
    }
}
```

```
@Override
public double berechneFläche()
{
    return Math.PI * radius * radius;
}
}
```

Erläuterung

- Kreis implementiert HatFläche.
- Die Fläche wird mit der Formel $\pi * r^2$ berechnet.

—

8. Klasse Rechteck

```
package template.fläche;

public class Rechteck implements HatFläche
{
    private double a;
    private double b;

    public Rechteck(double a, double b)
    {
        this.a = a;
        this.b = b;
    }

    public double getA()
    {
        return a;
    }

    public void setA(double a)
    {
        this.a = a;
    }
}
```

```

    public double getB()
    {
        return b;
    }

    public void setB(double b)
    {
        this.b = b;
    }

    @Override
    public double berechneFläche()
    {
        return a * b;
    }
}

```

Erläuterung

- Rechteck implementiert ebenfalls HatFläche.
- Die Fläche wird mit $a * b$ berechnet.

9. Testklasse Main

```

package template;

import template.drucken.*;
import template.fläche.HatFläche;
import template.fläche.Kreis;
import template.fläche.Rechteck;

import java.util.ArrayList;
import java.util.List;

public class Main
{
    public static void main(String[] args)

```

```

{
    // Statische Methode des Interfaces aufrufen
    Druckbar.testDruck();

    System.out.println();

    // Dokument erzeugen und default-Methode
    verwenden
    Dokument dokument = new Dokument("Hallo
    Welt!");
    if (dokument.prüfeFormat("297 mm x 210 mm"))
    {
        dokument.drucken();
    }

    System.out.println();

    // PDFDokument über den Typ PDFDruckbar
    ansprechen
    PDFDruckbar pdfDruckbar = new
    PDFDokument("Hallo Welt! (in einem PDF Dokument)");
    pdfDruckbar.drucken();
    pdfDruckbar.pdfDrucken();

    System.out.println();

    // PDFDokument über den Typ Mailbar
    ansprechen
    Mailbar mailbar = new PDFDokument("Hallo
    Welt! (wollen wir als Mail versenden)");
    mailbar.mailVersenden();

    System.out.println();

    // Liste mit Interface-Typ
    List<Druckbar> liste = new ArrayList<>();
    liste.add(dokument);
    liste.add(pdfDruckbar);
    liste.add((PDFDokument) mailbar);

    for (Druckbar d : liste)

```



```

        {
            d.drucken();
        }

        System.out.println();

        // Flächenbeispiele
        Rechteck rechteck = new Rechteck(5, 10);
        Kreis kreis = new Kreis(2);

        System.out.println("Ausgabe Rechteck:");
        ausgabeFläche(rechteck);

        System.out.println("Ausgabe Kreis:");
        ausgabeFläche(kreis);
    }

    static void ausgabeFläche(HatFläche hatFläche)
    {
        System.out.println("Fläche: " +
            hatFläche.berechneFläche());
    }
}

```

10. Analyse der Interface-Methoden

Abstrakte Methode

Beispiel:

```
void drucken();
```

- Diese Methode hat im Interface keine konkrete Implementierung.
- Jede Klasse, die Druckbar implementiert, muss diese Methode

umsetzen.

Default-Methode

Beispiel:

```
default boolean prüfeFormat(String format)
{
    return format.equalsIgnoreCase(DINA4);
}
```

- Diese Methode besitzt bereits eine Implementierung.
- Implementierende Klassen können sie direkt verwenden.

Statische Methode

Beispiel:

```
static void testDruck()
{
    System.out.println("Druckergeräusche... es wird  
ein Testblatt gedruckt.");
}
```

- Diese Methode gehört zum Interface selbst.
- Sie wird mit `Druckbar.testDruck()` aufgerufen.

11. Analyse der Typbeziehungen

Warum kann von einem Interface kein Objekt erzeugt werden?

Ein Interface beschreibt nur, **welche Methoden vorhanden sein müssen**, aber nicht vollständig, **wie ein konkretes Objekt aufgebaut ist**. Deshalb kann man nicht schreiben:

```
// Druckbar d = new Druckbar();    // nicht erlaubt
```

Warum kann Dokument als Druckbar betrachtet werden?

Weil Dokument das Interface Druckbar implementiert:

```
public class Dokument implements Druckbar
```

Damit erfüllt Dokument den Vertrag von Druckbar.

Warum kann PDFDokument gleichzeitig Druckbar, PDFDruckbar und Mailbar sein?

Weil:

- PDFDokument von Dokument erbt,
- Dokument bereits Druckbar implementiert,
- PDFDokument zusätzlich PDFDruckbar und Mailbar implementiert.

Dadurch kann dasselbe Objekt über verschiedene Typen betrachtet werden.

Warum können Kreis und Rechteck an `ausgabeFläche(HatFläche hatFläche)` übergeben werden?

Weil beide Klassen `HatFläche` implementieren. Die Methode erwartet also nur, dass das übergebene Objekt eine `berechneFläche()`-Methode besitzt.

Vorteil von Interface-Typen

Die Verwendung von Interface-Typen macht Programme:

- flexibler,
- erweiterbarer,
- allgemeiner formuliert.

So kann eine Methode mit vielen verschiedenen Objekten arbeiten, solange diese das passende Interface implementieren.

Beispielausgabe

Eine mögliche Konsolenausgabe ist:

```
Druckergeräusche... es wird ein Testblatt gedruckt.
```

```
Druckergeräusche... Das Dokument wird gedruckt: Hallo Welt!
```

Druckergeräusche... Das Dokument wird gedruckt: Hallo Welt! (in einem PDF Dokument)

Digitale Druckergeräusche... Das Dokument wird als PDF gedruckt: Hallo Welt! (in einem PDF Dokument)

Digitale Briefgeräusche... Das Dokument wird per Mail versendet: Hallo Welt! (wollen wir als Mail versenden)

Druckergeräusche... Das Dokument wird gedruckt: Hallo Welt!

Druckergeräusche... Das Dokument wird gedruckt: Hallo Welt! (in einem PDF Dokument)

Druckergeräusche... Das Dokument wird gedruckt: Hallo Welt! (wollen wir als Mail versenden)

Ausgabe Rechteck:

Fläche: 50.0

Ausgabe Kreis:

Fläche: 12.566370614359172

Zusammenfassung

Diese Musterlösung zeigt:

- ein grundlegendes Interface (Druckbar),
- ein erweitertes Interface (PDFDruckbar),
- ein weiteres unabhängiges Interface (Mailbar),
- konkrete Klassen, die Interfaces implementieren,
- die Kombination aus Vererbung und Interface-Implementierung,
- ein zweites Beispiel mit HatFläche, Kreis und Rechteck,

- die polymorphe Nutzung von Interfaces in Variablen, Listen und Methodenparametern.